
CNN pipeline on 1024×1024 input with 4×4 kernel

Chetan Somana / csomana

Design split into three stages

1. Convolution
2. Leaky ReLU
3. Pooling

The Kernel and input data of 1024×1024 are required for stage 1, convolution.

Challenges A

1. How the design issues read command and read addr to DRAM input memory.
2. How does the design stores the output of DRAM input memory.
3. In which pattern do we slide?
4. How big is the register file?

Each slide operation will populate convolution matrix.

Challenges B

1. Where do I store MAC output?
2. How big is my convolution register file

Leaky ReLU

Challenges C

1. How big is my Leaky ReLU register file
2. Modifying the weights based on if the number is positive/negative.

Pooling

Challenges D

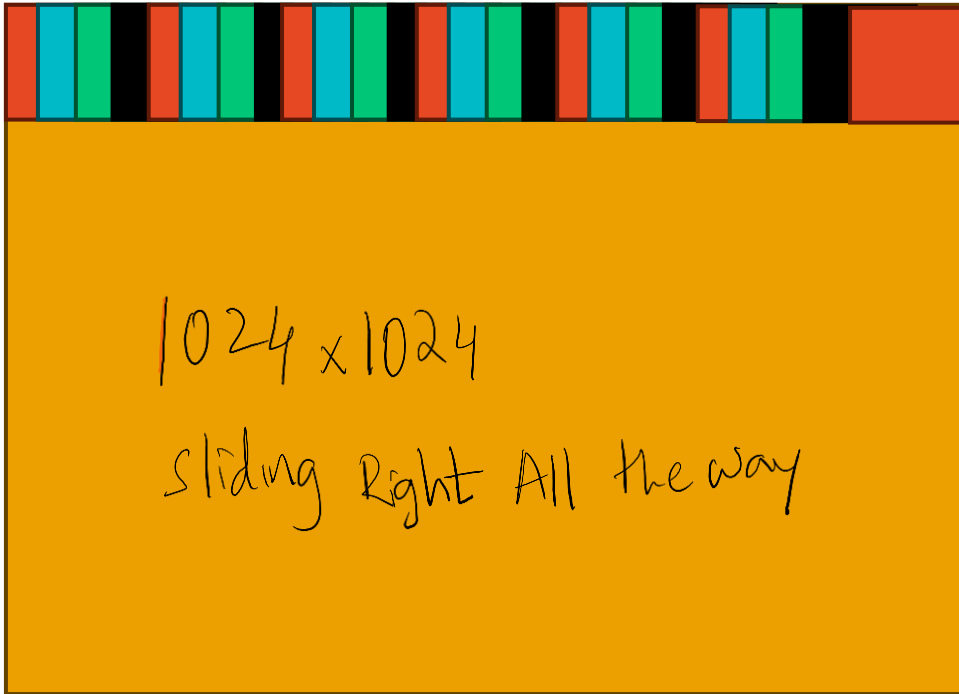
1. How big is my Pooling register file

Decisions made in Challenges A influence other Challenges

Challenge A -3 "In which pattern do we slide?"

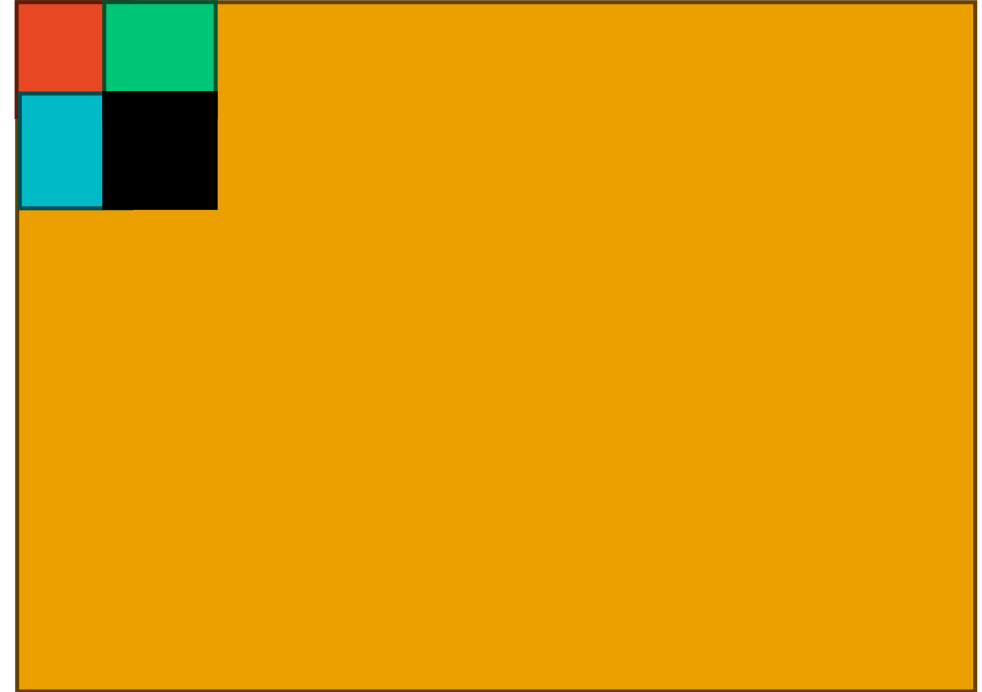
I can think of these two possibilities for this

First Possibility : Red ->blue->green->black



Sliding one way all the way 1021 times generates 1021 convolution elements. We need 4 elements in a 2*2 grid to perform average pool. But we have all the elements in for just one row. With this methods my convolution register file should be at least 1021B or more

Second Possibility Red ->shift down blue-> shift up green -> down black



If we want to limit the register file of convolution to 4B then we need to slide first at $(x,y) \rightarrow (x,y+1) \rightarrow (x+1,y) \rightarrow (x+1,y+1)$

Challenge A -3 solves Challenge B-2, C-2, D-1

Convolution register File 4B. Need to think if I should declare as reg [7:0] con_reg_file[2][2] or reg [7:0] con_reg_file[4]

Do I keep separate Leaky ReLU or overwrite the convolution register file ? If I have a separate Leaky ReLU one extra cycle is needed. Leaky ReLU reg file 4B.

Pooling reg file 8B

Challenges A

1. How the design issues read command and read addr to DRAM input memory.
2. How does the design stores the output of DRAM input memory.
- ~~3. In which pattern do we slide?~~
4. How big is the register file?

Challenges B

1. Where do I store MAC output?
- ~~2. How big is my convolution register file~~

Challenges C

- ~~1. How big is my Leaky ReLU register file~~
2. Modifying the weights based on if the number is positive/negative.

Challenges D

- ~~1. How big is my Pooling register file~~

As we are sliding in inverted N pattern, we need to have data from 5 adjacent rows.

Address 0 and 8 are occupied by the kernel in the DRAM input memory.

1024 * 1024 matrix is stored in the DRAM input memory.

Starting address for a particular row : $16 + 128 * (\text{\#row} - 1) * 8$

Ending address for a particular row: starting address + 127 * 8

ROWS

Kernel	1	2	3	4	5
--------	---	---	---	---	---

Read address 0->8->16->1040->2064->3088->4112

Design clock 0->8->16->24->32->40 ->48 ->56

Data capture 5->13->21->29->37 ->45 ->53->61 [5 cycle latency]

Data availability 13->21->29->37 ->45 ->53 ->61->69

First slide start at 69

Addr DATA

16	1	2	3	4	5	6	7	8
1040	1	2	3	4	5	6	7	8
2064	1	2	3	4	5	6	7	8
3088	1	2	3	4	5	6	7	8
4112	1	2	3	4	5	6	7	8

16	1	2	3	4	5	6	7	8
1040	1	2	3	4	5	6	7	8
2064	1	2	3	4	5	6	7	8
3088	1	2	3	4	5	6	7	8
4112	1	2	3	4	5	6	7	8

16	1	2	3	4	5	6	7	8
1040	1	2	3	4	5	6	7	8
2064	1	2	3	4	5	6	7	8
3088	1	2	3	4	5	6	7	8
4112	1	2	3	4	5	6	7	8

16	1	2	3	4	5	6	7	8
1040	1	2	3	4	5	6	7	8
2064	1	2	3	4	5	6	7	8
3088	1	2	3	4	5	6	7	8
4112	1	2	3	4	5	6	7	8

Addr DATA

24	1	2	3	4	5	6	7	8
1048	1	2	3	4	5	6	7	8
2072	1	2	3	4	5	6	7	8
3096	1	2	3	4	5	6	7	8
4120	1	2	3	4	5	6	7	8

24	1	2	3	4	5	6	7	8
1048	1	2	3	4	5	6	7	8
2072	1	2	3	4	5	6	7	8
3096	1	2	3	4	5	6	7	8
4120	1	2	3	4	5	6	7	8

24	1	2	3	4	5	6	7	8
1048	1	2	3	4	5	6	7	8
2072	1	2	3	4	5	6	7	8
3096	1	2	3	4	5	6	7	8
4120	1	2	3	4	5	6	7	8

24	1	2	3	4	5	6	7	8
1048	1	2	3	4	5	6	7	8
2072	1	2	3	4	5	6	7	8
3096	1	2	3	4	5	6	7	8
4120	1	2	3	4	5	6	7	8

Slide 1

Slide 2

Slide 3

Slide 4

Slide 5

16	1	2	3	4	5	6	7	8
1040	1	2	3	4	5	6	7	8
2064	1	2	3	4	5	6	7	8
3088	1	2	3	4	5	6	7	8
4112	1	2	3	4	5	6	7	8

24	1	2	3	4	5	6	7	8
1048	1	2	3	4	5	6	7	8
2072	1	2	3	4	5	6	7	8
3096	1	2	3	4	5	6	7	8
4120	1	2	3	4	5	6	7	8

Slide 6

16	1	2	3	4	5	6	7	8
1040	1	2	3	4	5	6	7	8
2064	1	2	3	4	5	6	7	8
3088	1	2	3	4	5	6	7	8
4112	1	2	3	4	5	6	7	8

24	1	2	3	4	5	6	7	8
1048	1	2	3	4	5	6	7	8
2072	1	2	3	4	5	6	7	8
3096	1	2	3	4	5	6	7	8
4120	1	2	3	4	5	6	7	8

Slide 7

16	1	2	3	4	5	6	7	8
1040	1	2	3	4	5	6	7	8
2064	1	2	3	4	5	6	7	8
3088	1	2	3	4	5	6	7	8
4112	1	2	3	4	5	6	7	8

24	1	2	3	4	5	6	7	8
1048	1	2	3	4	5	6	7	8
2072	1	2	3	4	5	6	7	8
3096	1	2	3	4	5	6	7	8
4120	1	2	3	4	5	6	7	8

Slide 8

16	1	2	3	4	5	6	7	8
1040	1	2	3	4	5	6	7	8
2064	1	2	3	4	5	6	7	8
3088	1	2	3	4	5	6	7	8
4112	1	2	3	4	5	6	7	8

24	1	2	3	4	5	6	7	8
1048	1	2	3	4	5	6	7	8
2072	1	2	3	4	5	6	7	8
3096	1	2	3	4	5	6	7	8
4120	1	2	3	4	5	6	7	8

Slide 9

16	1	2	3	4	5	6	7	8
1040	1	2	3	4	5	6	7	8
2064	1	2	3	4	5	6	7	8
3088	1	2	3	4	5	6	7	8
4112	1	2	3	4	5	6	7	8

24	1	2	3	4	5	6	7	8
1048	1	2	3	4	5	6	7	8
2072	1	2	3	4	5	6	7	8
3096	1	2	3	4	5	6	7	8
4120	1	2	3	4	5	6	7	8

Slide 10

16	1	2	3	4	5	6	7	8
1040	1	2	3	4	5	6	7	8
2064	1	2	3	4	5	6	7	8
3088	1	2	3	4	5	6	7	8
4112	1	2	3	4	5	6	7	8

24	1	2	3	4	5	6	7	8
1048	1	2	3	4	5	6	7	8
2072	1	2	3	4	5	6	7	8
3096	1	2	3	4	5	6	7	8
4120	1	2	3	4	5	6	7	8

Slide 11

16	1	2	3	4	5	6	7	8
1040	1	2	3	4	5	6	7	8
2064	1	2	3	4	5	6	7	8
3088	1	2	3	4	5	6	7	8
4112	1	2	3	4	5	6	7	8

24	1	2	3	4	5	6	7	8
1048	1	2	3	4	5	6	7	8
2072	1	2	3	4	5	6	7	8
3096	1	2	3	4	5	6	7	8
4120	1	2	3	4	5	6	7	8

Slide 12

16	1	2	3	4	5	6	7	8
1040	1	2	3	4	5	6	7	8
2064	1	2	3	4	5	6	7	8
3088	1	2	3	4	5	6	7	8
4112	1	2	3	4	5	6	7	8

24	1	2	3	4	5	6	7	8
1048	1	2	3	4	5	6	7	8
2072	1	2	3	4	5	6	7	8
3096	1	2	3	4	5	6	7	8
4120	1	2	3	4	5	6	7	8

Slide 13

16	1	2	3	4	5	6	7	8		24	1	2	3	4	5	6	7	8
1040	1	2	3	4	5	6	7	8		1048	1	2	3	4	5	6	7	8
2064	1	2	3	4	5	6	7	8		2072	1	2	3	4	5	6	7	8
3088	1	2	3	4	5	6	7	8		3096	1	2	3	4	5	6	7	8
4112	1	2	3	4	5	6	7	8		4120	1	2	3	4	5	6	7	8

Slide 14

16	1	2	3	4	5	6	7	8		24	1	2	3	4	5	6	7	8
1040	1	2	3	4	5	6	7	8		1048	1	2	3	4	5	6	7	8
2064	1	2	3	4	5	6	7	8		2072	1	2	3	4	5	6	7	8
3088	1	2	3	4	5	6	7	8		3096	1	2	3	4	5	6	7	8
4112	1	2	3	4	5	6	7	8		4120	1	2	3	4	5	6	7	8

Slide 15

16	1	2	3	4	5	6	7	8		24	1	2	3	4	5	6	7	8
1040	1	2	3	4	5	6	7	8		1048	1	2	3	4	5	6	7	8
2064	1	2	3	4	5	6	7	8		2072	1	2	3	4	5	6	7	8
3088	1	2	3	4	5	6	7	8		3096	1	2	3	4	5	6	7	8
4112	1	2	3	4	5	6	7	8		4120	1	2	3	4	5	6	7	8

Slide 16

16	1	2	3	4	5	6	7	8		24	1	2	3	4	5	6	7	8
1040	1	2	3	4	5	6	7	8		1048	1	2	3	4	5	6	7	8
2064	1	2	3	4	5	6	7	8		2072	1	2	3	4	5	6	7	8
3088	1	2	3	4	5	6	7	8		3096	1	2	3	4	5	6	7	8
4112	1	2	3	4	5	6	7	8		4120	1	2	3	4	5	6	7	8

5*8 Register file 1

16	1	2	3	4	5	6	7	8
1040	1	2	3	4	5	6	7	8
2064	1	2	3	4	5	6	7	8
3088	1	2	3	4	5	6	7	8
4112	1	2	3	4	5	6	7	8

5 * 8 Register file 2

24	1	2	3	4	5	6	7	8
1048	1	2	3	4	5	6	7	8
2072	1	2	3	4	5	6	7	8
3096	1	2	3	4	5	6	7	8
4120	1	2	3	4	5	6	7	8

2*8 highlighted elements in yellow can be removed as we have utilized them completely. Achieve by overwriting.
 3*8 highlighted elements in green moved to a buffer which is SRAM of size 32Kb or 3KB Register file.

Copy

After 15 slide is done, copy the first row from 5*8 RF2 to 5*8 RF1 so that 11th read can be stored
 In 5*8 RF2.

After 16 slide is done, wait till 10 read is completed and then copy next four rows from 5*8 RF2 to 5*8 RF1
 Give 12,13,14,15 reads while 11 read is not completed

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
1		KERNEL		ROWS											
2				5*8 buffer 1					5*8 buffer 2					5*8 buffer 1	
3				1	2	3	4	5	1	2	3	4	5	1	2
4	Read address	0	8	16	1040	2064	3088	4112	24	1048	2072	3096	4120	32	1056
5	Design Clock	0	8	16	24	32	40	48	56	64	72	80	88	96	
6	Data capture clock	5	13	21	29	37	45	53	61	69	77	85	93	101	
7	Data availability clock	13	21	28	36	44	52	60	68	76	84	82	100	108	
8	slide start clock								61	69			93		
9	#Accumulated Slide 1/cycle								1 -- 8	9 10			11--16		
10	revised slide start clock										83	85	93		
11	#Accumulated Slide 1/cycle										1--2	3--10	11--16		
12															

When the clock is at 98, i.e 15th slide is completed, generate control signal which triggers to copy RF2 row1 to RF1 row1.

When the 11th read data capturing begin, generate a control signal to copy RF2 row2-5 to RF1 row 2-5.

5*8 Register file 1

24	1	2	3	4	5	6	7	8
1048	1	2	3	4	5	6	7	8
2072	1	2	3	4	5	6	7	8
3096	1	2	3	4	5	6	7	8
4120	1	2	3	4	5	6	7	8

5 * 8 Register file 2

32	1	2	3	4	5	6	7	8
1056	1	2	3	4	5	6	7	8
2080	1	2	3	4	5	6	7	8
3104	1	2	3	4	5	6	7	8
4128	1	2	3	4	5	6	7	8

The new data always comes to RF2. Where as the the bottom 3 rows of RF1 is being sent to SRAM or 3KB register file(Not decided yet. Will change based on the synthesis/performance problems)

Row transitioning discussed on next slide

RF1 : Last read – 1 of the row

RF2: Last read of the row

RF1 : Last read of the row

RF2: New read of the next row(last row +5)

RF1 : New read of the next row(last row +5)

RF2: New read +1 of the next row(last row +5)

Challenges A

- ~~1. How the design issues read command and read addr to DRAM input memory.~~ Rows 1 to 5 till last read of the row
- ~~2. How does the design stores the output of DRAM input memory.~~ Two 5*8 RF and buffer unused RF data
- ~~3. In which pattern do we slide?~~ Inverted N
- ~~4. How big is the register file?~~ Two 5*8 RF

Challenges B

- ~~1. Where do I store MAC output?~~ In convolution reg file
- ~~2. How big is my convolution register file~~ 4B

Challenges C

- ~~1. How big is my Leaky ReLU register file~~ 4B
- ~~2. Modifying the weights based on if the number is positive/negative.~~

Challenges D

- ~~1. How big is my Pooling register file~~ 8B

5 rows -> 5 read counter. Each row initialized
After reset, FSM always transitions from IDLE to READ LOAD

IDLE : 5 read counter are initialized to 0

READ LOAD :

If number of rows_reads == 0 then

Row1_read_count = 16;

Else

Row1_read_count = 16 + 128*8*(3 + 5*(rows_read-1));

READ INCR: (if the 32 cycles has passed only then we should enter this state)

Row1_read_count = Row1_read_count + 8;

Row2_read_count = Row1_read_count + 1024;

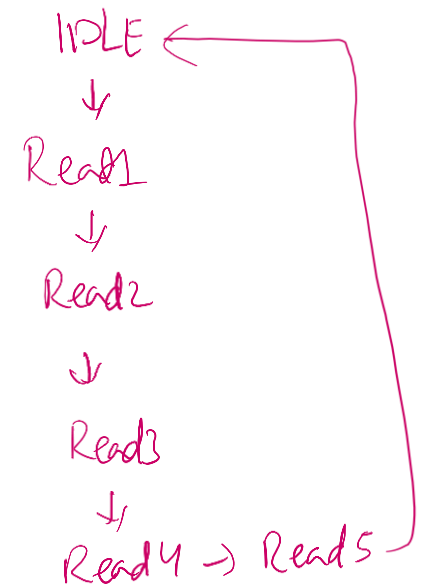
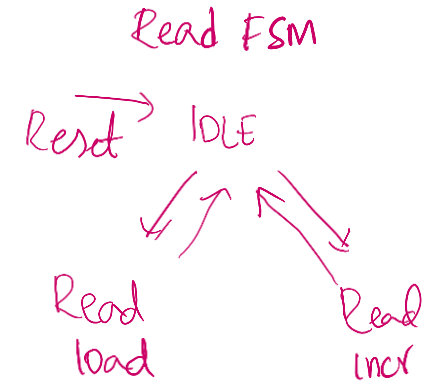
Row3_read_count = Row2_read_count + 1024;

Row4_read_count = Row3_read_count + 1024;

Row5_read_count = Row4_read_count + 1024;

Need extra logic to choose which row*_read_count should be sent on addr of input DRAM
6 bit counter is sufficient as we need to count till 40 cycles.

Decide between counter or FSM with 6 states and a 3bit counter to goto next state.



Writes to output DRAM

1. Convolution buffer is filled in row order form, thus decreasing complexity on write.
2. Write address start at 0. Increment write address by 8.
3. Whenever pooling buffer is accessed assert a pooling_buffer_accessed signal.
4. Count pooling_buffer_accessed signal, when it reaches 7 trigger write and

Control signals identified

1. last_read : Keep tracks of how many reads for a row has undergone.
2. two_rows_used : is set when the first two rows of the input matrix is utilized
3. swapped : is set when RF2 contents is swapped to RF1, this information is useful for sliding.
4. Relu start : indicates 4 sliding has been done in inverted N pattern. Convolution matrix is ready with 4 elements.
5. write_buffer : is set when there are 8 bytes of information which is ready to write to output DRAM
6. slide enable : perform slide only when this bit is set high
7. First_row_swap : is set when RF2 row1 can be copied to RF1 row1. Usually set for after every 10th read
8. 16_slides_dones : indicates 16 convolution data is generated, (write_buffer should go with in few clock cycles)

Slide 1

16	1	2	3	4	5	6	7	8
1040	1	2	3	4	5	6	7	8
2064	1	2	3	4	5	6	7	8
3088	1	2	3	4	5	6	7	8
4112	1	2	3	4	5	6	7	8

Slide 2

16	1	2	3	4	5	6	7	8
1040	1	2	3	4	5	6	7	8
2064	1	2	3	4	5	6	7	8
3088	1	2	3	4	5	6	7	8
4112	1	2	3	4	5	6	7	8

Slide 3

16	1	2	3	4	5	6	7	8
1040	1	2	3	4	5	6	7	8
2064	1	2	3	4	5	6	7	8
3088	1	2	3	4	5	6	7	8
4112	1	2	3	4	5	6	7	8

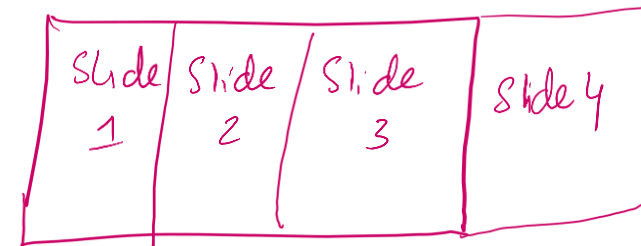
Slide 4

16	1	2	3	4	5	6	7	8
1040	1	2	3	4	5	6	7	8
2064	1	2	3	4	5	6	7	8
3088	1	2	3	4	5	6	7	8
4112	1	2	3	4	5	6	7	8

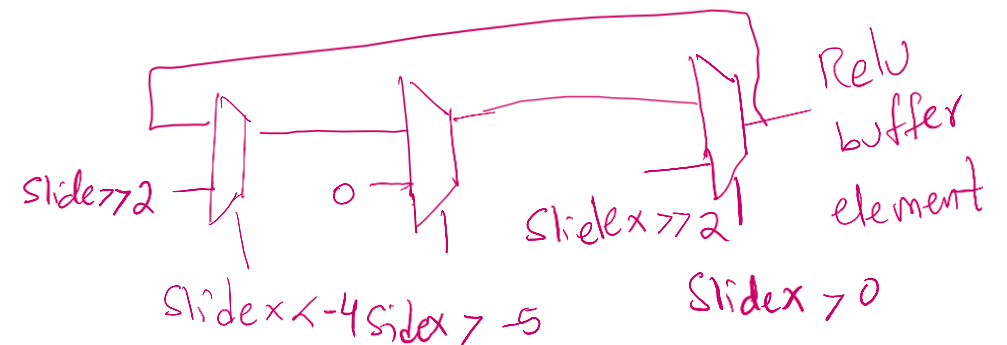
16 multiplier

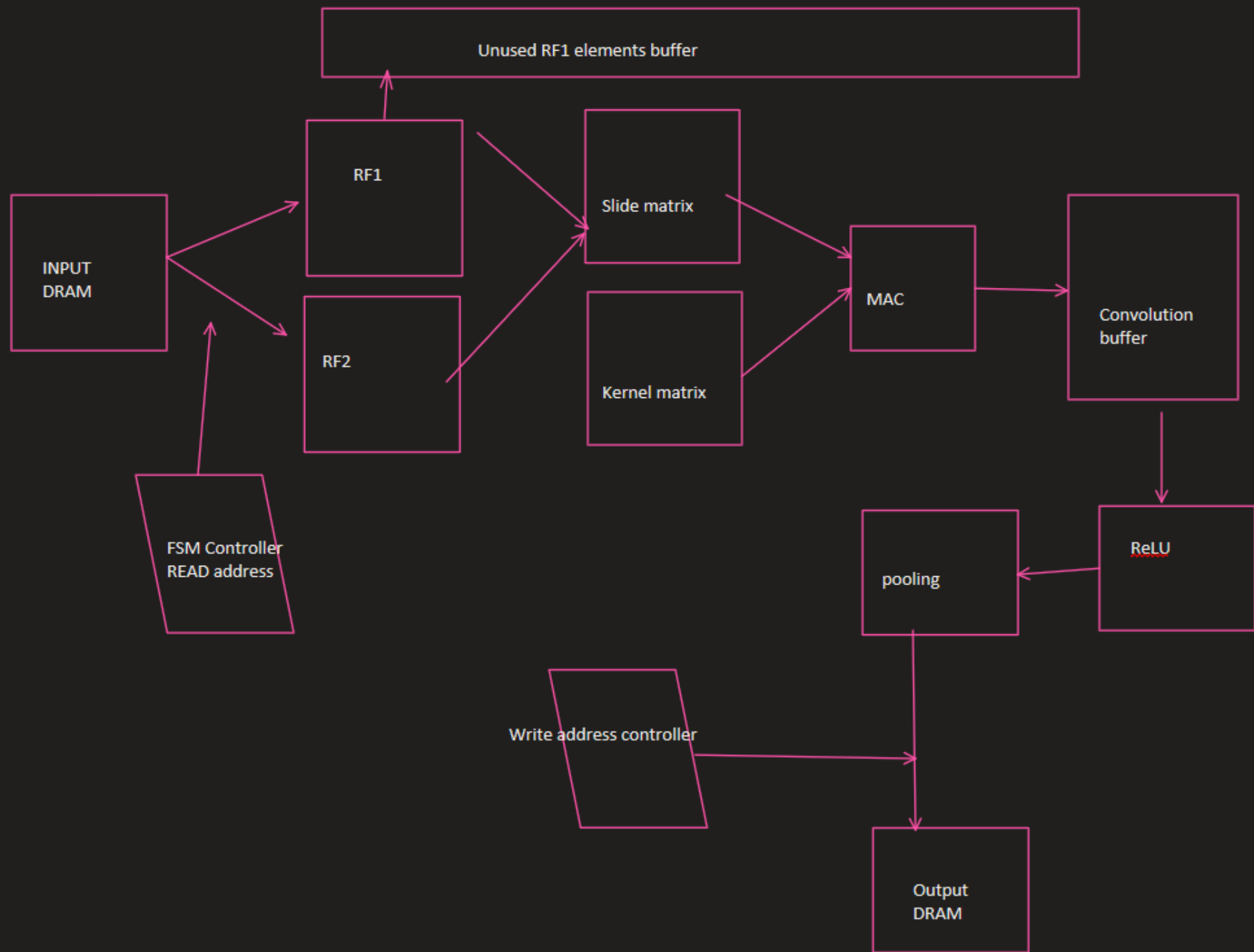
16 adder

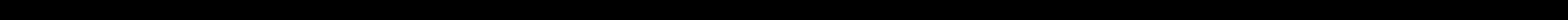
And result place in convolution buffer



ReLU buffer







Storage / Controller elements identified in the design

1. Two 5*8 to store data from output DRAM
2. 3*1024 or SRAM to transfer back unused data (decide based on synthesis/performance)
3. 4B Convolution Buffer
4. 4B Leaky ReLU Buffer
5. 8B Pooling buffer

6. Read address generator logic.
7. Sliding logic
8. MAC logic
9. Register swap logic
10. Write address generator logic

Make the flag 1 when RF2 contents are swapped to RF1. When this flag is set, sliding should always start from 4 location from the left as the previous elements are Used in earlier slides. Process repeated till slide 2047,2048 before switching to the next row

Slide 17

24	1	2	3	4	5	6	7	8	32	1	2	3	4	5	6	7	8
1048	1	2	3	4	5	6	7	8	1056	1	2	3	4	5	6	7	8
2072	1	2	3	4	5	6	7	8	2080	1	2	3	4	5	6	7	8
3096	1	2	3	4	5	6	7	8	3104	1	2	3	4	5	6	7	8
4120	1	2	3	4	5	6	7	8	4128	1	2	3	4	5	6	7	8

Slide 18

24	1	2	3	4	5	6	7	8	32	1	2	3	4	5	6	7	8
1048	1	2	3	4	5	6	7	8	1056	1	2	3	4	5	6	7	8
2072	1	2	3	4	5	6	7	8	2080	1	2	3	4	5	6	7	8
3096	1	2	3	4	5	6	7	8	3104	1	2	3	4	5	6	7	8
4120	1	2	3	4	5	6	7	8	4128	1	2	3	4	5	6	7	8

Slide 19

24	1	2	3	4	5	6	7	8	32	1	2	3	4	5	6	7	8
1048	1	2	3	4	5	6	7	8	1056	1	2	3	4	5	6	7	8
2072	1	2	3	4	5	6	7	8	2080	1	2	3	4	5	6	7	8
3096	1	2	3	4	5	6	7	8	3104	1	2	3	4	5	6	7	8
4120	1	2	3	4	5	6	7	8	4128	1	2	3	4	5	6	7	8

Slide 20

24	1	2	3	4	5	6	7	8	32	1	2	3	4	5	6	7	8
1048	1	2	3	4	5	6	7	8	1056	1	2	3	4	5	6	7	8
2072	1	2	3	4	5	6	7	8	2080	1	2	3	4	5	6	7	8
3096	1	2	3	4	5	6	7	8	3104	1	2	3	4	5	6	7	8
4120	1	2	3	4	5	6	7	8	4128	1	2	3	4	5	6	7	8

Slide 21

24	1	2	3	4	5	6	7	8	32	1	2	3	4	5	6	7	8
1048	1	2	3	4	5	6	7	8	1056	1	2	3	4	5	6	7	8
2072	1	2	3	4	5	6	7	8	2080	1	2	3	4	5	6	7	8
3096	1	2	3	4	5	6	7	8	3104	1	2	3	4	5	6	7	8
4120	1	2	3	4	5	6	7	8	4128	1	2	3	4	5	6	7	8

Slide 22

24	1	2	3	4	5	6	7	8	32	1	2	3	4	5	6	7	8
1048	1	2	3	4	5	6	7	8	1056	1	2	3	4	5	6	7	8
2072	1	2	3	4	5	6	7	8	2080	1	2	3	4	5	6	7	8
3096	1	2	3	4	5	6	7	8	3104	1	2	3	4	5	6	7	8
4120	1	2	3	4	5	6	7	8	4128	1	2	3	4	5	6	7	8

Slide 23

24	1	2	3	4	5	6	7	8	32	1	2	3	4	5	6	7	8
1048	1	2	3	4	5	6	7	8	1056	1	2	3	4	5	6	7	8
2072	1	2	3	4	5	6	7	8	2080	1	2	3	4	5	6	7	8
3096	1	2	3	4	5	6	7	8	3104	1	2	3	4	5	6	7	8
4120	1	2	3	4	5	6	7	8	4128	1	2	3	4	5	6	7	8

Slide 24

24	1	2	3	4	5	6	7	8	32	1	2	3	4	5	6	7	8
1048	1	2	3	4	5	6	7	8	1056	1	2	3	4	5	6	7	8
2072	1	2	3	4	5	6	7	8	2080	1	2	3	4	5	6	7	8
3096	1	2	3	4	5	6	7	8	3104	1	2	3	4	5	6	7	8
4120	1	2	3	4	5	6	7	8	4128	1	2	3	4	5	6	7	8

Slide 25

24	1	2	3	4	5	6	7	8	32	1	2	3	4	5	6	7	8
1048	1	2	3	4	5	6	7	8	1056	1	2	3	4	5	6	7	8
2072	1	2	3	4	5	6	7	8	2080	1	2	3	4	5	6	7	8
3096	1	2	3	4	5	6	7	8	3104	1	2	3	4	5	6	7	8
4120	1	2	3	4	5	6	7	8	4128	1	2	3	4	5	6	7	8

Slide 26

24	1	2	3	4	5	6	7	8	32	1	2	3	4	5	6	7	8
1048	1	2	3	4	5	6	7	8	1056	1	2	3	4	5	6	7	8
2072	1	2	3	4	5	6	7	8	2080	1	2	3	4	5	6	7	8
3096	1	2	3	4	5	6	7	8	3104	1	2	3	4	5	6	7	8
4120	1	2	3	4	5	6	7	8	4128	1	2	3	4	5	6	7	8

Slide 27

24	1	2	3	4	5	6	7	8	32	1	2	3	4	5	6	7	8
1048	1	2	3	4	5	6	7	8	1056	1	2	3	4	5	6	7	8
2072	1	2	3	4	5	6	7	8	2080	1	2	3	4	5	6	7	8
3096	1	2	3	4	5	6	7	8	3104	1	2	3	4	5	6	7	8
4120	1	2	3	4	5	6	7	8	4128	1	2	3	4	5	6	7	8

Slide 28

24	1	2	3	4	5	6	7	8	32	1	2	3	4	5	6	7	8
1048	1	2	3	4	5	6	7	8	1056	1	2	3	4	5	6	7	8
2072	1	2	3	4	5	6	7	8	2080	1	2	3	4	5	6	7	8
3096	1	2	3	4	5	6	7	8	3104	1	2	3	4	5	6	7	8
4120	1	2	3	4	5	6	7	8	4128	1	2	3	4	5	6	7	8

Slide 25

24	1	2	3	4	5	6	7	8
1048	1	2	3	4	5	6	7	8
2072	1	2	3	4	5	6	7	8
3096	1	2	3	4	5	6	7	8
4120	1	2	3	4	5	6	7	8

32	1	2	3	4	5	6	7	8
1056	1	2	3	4	5	6	7	8
2080	1	2	3	4	5	6	7	8
3104	1	2	3	4	5	6	7	8
4128	1	2	3	4	5	6	7	8

Slide 26

24	1	2	3	4	5	6	7	8
1048	1	2	3	4	5	6	7	8
2072	1	2	3	4	5	6	7	8
3096	1	2	3	4	5	6	7	8
4120	1	2	3	4	5	6	7	8

32	1	2	3	4	5	6	7	8
1056	1	2	3	4	5	6	7	8
2080	1	2	3	4	5	6	7	8
3104	1	2	3	4	5	6	7	8
4128	1	2	3	4	5	6	7	8

Slide 27

24	1	2	3	4	5	6	7	8
1048	1	2	3	4	5	6	7	8
2072	1	2	3	4	5	6	7	8
3096	1	2	3	4	5	6	7	8
4120	1	2	3	4	5	6	7	8

32	1	2	3	4	5	6	7	8
1056	1	2	3	4	5	6	7	8
2080	1	2	3	4	5	6	7	8
3104	1	2	3	4	5	6	7	8
4128	1	2	3	4	5	6	7	8

Slide 28

24	1	2	3	4	5	6	7	8
1048	1	2	3	4	5	6	7	8
2072	1	2	3	4	5	6	7	8
3096	1	2	3	4	5	6	7	8
4120	1	2	3	4	5	6	7	8

32	1	2	3	4	5	6	7	8
1056	1	2	3	4	5	6	7	8
2080	1	2	3	4	5	6	7	8
3104	1	2	3	4	5	6	7	8
4128	1	2	3	4	5	6	7	8

Every row should issue 128 reads to fill 1024 locations. Maintain a counter when 127 is hit, we know it's the last read and
We should switch to the next row

Slide 29

24	1	2	3	4	5	6	7	8
1048	1	2	3	4	5	6	7	8
2072	1	2	3	4	5	6	7	8
3096	1	2	3	4	5	6	7	8
4120	1	2	3	4	5	6	7	8

32	1	2	3	4	5	6	7	8
1056	1	2	3	4	5	6	7	8
2080	1	2	3	4	5	6	7	8
3104	1	2	3	4	5	6	7	8
4128	1	2	3	4	5	6	7	8

Slide 30

24	1	2	3	4	5	6	7	8
1048	1	2	3	4	5	6	7	8
2072	1	2	3	4	5	6	7	8
3096	1	2	3	4	5	6	7	8
4120	1	2	3	4	5	6	7	8

32	1	2	3	4	5	6	7	8
1056	1	2	3	4	5	6	7	8
2080	1	2	3	4	5	6	7	8
3104	1	2	3	4	5	6	7	8
4128	1	2	3	4	5	6	7	8

Slide 31

24	1	2	3	4	5	6	7	8
1048	1	2	3	4	5	6	7	8
2072	1	2	3	4	5	6	7	8
3096	1	2	3	4	5	6	7	8
4120	1	2	3	4	5	6	7	8

32	1	2	3	4	5	6	7	8
1056	1	2	3	4	5	6	7	8
2080	1	2	3	4	5	6	7	8
3104	1	2	3	4	5	6	7	8
4128	1	2	3	4	5	6	7	8

Slide 32

24	1	2	3	4	5	6	7	8
1048	1	2	3	4	5	6	7	8
2072	1	2	3	4	5	6	7	8
3096	1	2	3	4	5	6	7	8
4120	1	2	3	4	5	6	7	8

32	1	2	3	4	5	6	7	8
1056	1	2	3	4	5	6	7	8
2080	1	2	3	4	5	6	7	8
3104	1	2	3	4	5	6	7	8
4128	1	2	3	4	5	6	7	8



